

1. AI-Powered Drone for Emergency Medicine Delivery (Greedy Best-First Search & A*)

Problem Statement

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is dynamic due to blocked roads, changing weather, and variable travel costs.

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the next path based only on the heuristic value $h(n)$ (estimated distance to the goal).

Working:

- Chooses the node that appears closest to the destination.
- Ignores the actual travel cost.

Strengths:

- Very fast.
- Low memory usage.
- Suitable for urgent situations requiring quick decisions.

Weaknesses:

- Does not guarantee the shortest or safest path.
- May choose risky routes.
- Performs poorly when blocked paths appear suddenly.
- Can get trapped in dead ends.

ii. A* Search Algorithm

A* uses both actual cost and heuristic value.

Formula:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = Actual travel cost from the start.
- $h(n)$ = Estimated cost to reach the goal.

Working:

- Balances travel cost and estimated distance.
- Recalculates routes when obstacles appear.
- Finds the optimal path if the heuristic is admissible.

Advantages:

- Produces safer routes.
- Handles varying terrain and weather.
- More reliable than GBFS.

iii. Impact of Heuristic Accuracy

Accurate Heuristic

- GBFS reaches the goal faster.
- A* explores fewer nodes and remains optimal.

Poor Heuristic

- GBFS may follow wrong paths.
- A* still finds the best solution but takes longer.

iv. Effect of Dynamic Changes

When roads become blocked:

- GBFS often restarts the search.
- A* updates the path considering the new cost and obstacles.

Dynamic environments increase search time and computational effort.

v. Recommendation

Recommended Algorithm: A*

Justification

- Produces optimal paths.
- Considers travel cost and safety.
- Adapts to blocked paths.
- Suitable for emergency medicine delivery where safety is critical.

2. Smart City Traffic Signal Optimization (Hill Climbing & Optimization)

Problem Formulation

Objective

Minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Why Traditional Search is Inefficient

- Huge search space.
- Traffic changes continuously.
- Searching every possible signal timing is computationally impossible.

i. Hill Climbing

Hill Climbing starts with an initial solution and repeatedly selects a neighboring solution with better performance.

Advantages

- Simple.
- Fast.
- Low memory requirement.

Limitations

- Gets stuck in local optimum.
- Cannot escape flat regions.

- May stop before finding the best solution.

ii. Local Maxima, Plateaus, and Ridges

Local Maxima

- Signal timings appear optimal but are not globally best.

Plateau

- Many neighboring solutions have the same quality.
- Search cannot determine which direction to move.

Ridge

- Best improvement requires several coordinated changes.
- Hill Climbing struggles to follow the correct path.

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing

- Occasionally accepts worse solutions.
- Escapes local maxima.
- Eventually converges to a good solution.

Genetic Algorithm

- Maintains multiple candidate solutions.
- Uses selection, crossover, and mutation.
- Explores many regions simultaneously.
- Highly suitable for complex optimization.

iv. Recommendation

Recommended Approach: Genetic Algorithm

Reasons

- Excellent scalability.
- Handles dynamic traffic conditions.
- Avoids local optima.
- Produces near-optimal solutions for large search spaces.

3. Autonomous Mars Rover (Online Search Agent)

i. Why Online Search?

The rover has:

- Unknown terrain.
- Limited sensors.
- No complete map.

Therefore, decisions must be made while exploring.

Offline search is impossible because the entire environment is unknown.

ii. Challenges

Partial Observability

- The rover sees only nearby terrain.

Dynamic Environment

- Dust storms.
- New obstacles.
- Changing terrain conditions.

iii. Internal Model

The rover:

- Collects sensor data.
- Updates the map continuously.

- Marks hazards.
- Stores safe routes.
- Improves navigation after every movement.

iv. Comparison

Uninformed Search

- No heuristic.
- Explores many unnecessary nodes.

Informed Search

- Uses heuristic information.
- Faster than uninformed search.

Online Search

- Learns while moving.
- Updates decisions continuously.
- Best for unknown environments.

v. Recommendation

Recommended Approach: Online Search Agent

Reasons

- Handles uncertainty.
- Learns continuously.
- Adapts to new hazards.
- Ensures safer navigation

4. University Exam Timetable (Constraint Satisfaction Problem - CSP)

Problem Formulation

Variables

Each exam represents one variable.

Example:

- AI Exam
- DBMS Exam
- OS Exam
- CN Exam

Domains

Possible:

- Dates
- Time slots
- Exam halls

Constraints

- No student has overlapping exams.
- Hall capacity should not be exceeded.
- Faculty must be available.
- Some subjects must occur before others.
- Special accommodation students require suitable halls.

ii. Backtracking Search

Working:

1. Assign a time slot.
2. Check constraints.
3. If constraints fail, undo the assignment.
4. Try another time slot.
5. Continue until all exams are scheduled.

iii. Constraint Propagation (Forward Checking)

Forward Checking:

- Removes invalid future choices immediately.
- Detects conflicts early.
- Reduces unnecessary search.
- Speeds up timetable generation.

iv. Real-world Challenges

- Thousands of students.
- Hundreds of courses.
- Limited halls.
- Faculty scheduling.
- Last-minute changes.

Best Strategy

Use CSP with Backtracking and Forward Checking for efficient and scalable timetable generation.

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Minimax models the game as a decision tree.

MAX Player

Attempts to maximize its score.

MIN Player

Attempts to minimize the opponent's score.

The algorithm assumes both players play optimally.

ii. Computational Challenges

- Branching factor is very high.
- Millions of game states.

- High memory usage.
- Slow response in real-time games.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning removes branches that cannot affect the final decision.

Advantages

- Reduces nodes explored.
- Produces the same optimal result as Minimax.
- Improves execution speed.
- Suitable for real-time strategy games.

iv. Evaluation Function

The evaluation function estimates the quality of a game state.

Example factors:

- Player health.
- Number of units.
- Resources collected.
- Territory controlled.
- Distance from enemy.
- Strategic position.

A higher evaluation score indicates a stronger game state.

v. Recommendation

Recommended Strategy: Alpha-Beta Pruning with Minimax

Justification

- Maintains optimal decisions.
- Greatly reduces computation.
- Meets strict real-time constraints.

- Widely used in modern game AI.